
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 10

Estruturas, Uniões, Manipulações de Bits e Enumerações

Fazendo a Diferença

Exercício 10.21

Informatização de registros de saúde
Sistema integrado com IMC e frequência cardíaca

Contents

1	Exercício 10.21 – Perfil de Saúde	2
----------	--	----------

1. Exercício 10.21 – Perfil de Saúde

Enunciado 10.21

Crie uma estrutura `PerfilSaude` com nome, sobrenome, sexo, data de nascimento, altura (cm) e peso (kg). Implemente funções que:

- Recebam esses dados e definam os membros da estrutura.
- Calculem a idade em anos.
- Calculem a frequência cardíaca máxima e ideal.
- Calculem o IMC.

Exiba todas as informações junto com a tabela de classificação de IMC.

Pesquisa Externa

Pesquisa externa realizada

Frequência cardíaca máxima: fórmula clássica de Tanaka

$$FC_{\max} = 220 - \text{idade}$$

Frequência cardíaca ideal (zona aeróbica): 50% a 85% da $FC_{\max}$. A faixa típica para exercícios de queima de gordura é 60–70%; para condicionamento cardiovascular, 70–85%.

IMC: $IMC = \text{peso (kg)} / \text{altura (m)}^2$, com classificação da OMS (abaixo do peso, normal, sobrepeso, obeso) – a mesma do Ex. 2.32.

Cálculo de idade: diferença entre o ano atual e o ano de nascimento, *ajustada* se o aniversário ainda não ocorreu no ano corrente.

Solução em C

Resposta detalhada

```
1  /* Ex10.21: perfil_saude.c
2     Sistema integrado de perfil de saude */
3  #include <stdio.h>
4  #include <string.h>
5  #include <time.h>
6
7  /* Estrutura para data */
8  typedef struct {
9      int dia;
10     int mes;
11     int ano;
12 } Data;
13
14 /* Estrutura principal de perfil */
15 typedef struct {
16     char nome[ 30 ];
17     char sobrenome[ 30 ];
```

```
18     char sexo;                /* 'M' ou 'F' */
19     Data nascimento;
20     double altura_cm;
21     double peso_kg;
22 } PerfilSaude;
23
24 /* Prototipos */
25 void lerPerfil      ( PerfilSaude *p );
26 int  calcularIdade  ( const Data *nasc );
27 int  fcMaxima       ( int idade );
28 double fcIdealMin   ( int idade );
29 double fcIdealMax   ( int idade );
30 double calcularIMC   ( double peso_kg, double
31     altura_cm );
32 const char* classificarIMC( double imc );
33 void  exhibirPerfil ( const PerfilSaude *p );
34
35 int main( void )
36 {
37     PerfilSaude paciente;
38
39     printf( "==== SISTEMA DE PERFIL DE SAUDE =====\n\n" );
40     lerPerfil( &paciente );
41     exhibirPerfil( &paciente );
42
43     return 0;
44 }
45
46 void lerPerfil( PerfilSaude *p )
47 {
48     printf( "Nome: " );
49     scanf( "%29s", p->nome );
50
51     printf( "Sobrenome: " );
52     scanf( "%29s", p->sobrenome );
53
54     printf( "Sexo (M/F): " );
55     scanf( " %c", &p->sexo );
56
57     printf( "Data de nascimento (dia mes ano): " );
58     scanf( "%d%d%d", &p->nascimento.dia,
59         &p->nascimento.mes,
60         &p->nascimento.ano );
61
62     printf( "Altura em cm: " );
63     scanf( "%lf", &p->altura_cm );
64
65     printf( "Peso em kg: " );
66     scanf( "%lf", &p->peso_kg );
```

```
66 }
67
68 int calcularIdade( const Data *nasc )
69 {
70     time_t t = time( NULL );
71     struct tm hoje = *localtime( &t );
72
73     int anoAtual = hoje.tm_year + 1900;
74     int mesAtual = hoje.tm_mon + 1;
75     int diaAtual = hoje.tm_mday;
76
77     int idade = anoAtual - nasc->ano;
78
79     /* Se ainda nao fez aniversario, subtrai 1 */
80     if ( mesAtual < nasc->mes ||
81         ( mesAtual == nasc->mes && diaAtual < nasc->dia
82           ) ) {
83         idade--;
84     }
85     return idade;
86 }
87
88 int fcMaxima( int idade )
89 {
90     return 220 - idade;
91 }
92
93 double fcIdealMin( int idade )
94 {
95     return fcMaxima( idade ) * 0.50;
96 }
97
98 double fcIdealMax( int idade )
99 {
100     return fcMaxima( idade ) * 0.85;
101 }
102
103 double calcularIMC( double peso_kg, double altura_cm )
104 {
105     double altura_m = altura_cm / 100.0;
106     return peso_kg / ( altura_m * altura_m );
107 }
108
109 const char* classificarIMC( double imc )
110 {
111     if ( imc < 18.5 )         return "Abaixo do peso";
112     if ( imc < 25.0 )         return "Normal";
113     if ( imc < 30.0 )         return "Acima do peso (
114                               sobrepeso)";
115     return "Obeso";
116 }
```

```
114 }
115
116 void exibirPerfil( const PerfilSaude *p )
117 {
118     int idade = calcularIdade( &p->nascimento );
119     double imc = calcularIMC( p->peso_kg, p->altura_cm
120                             );
121
122     printf( "\n===== PERFIL =====\n" );
123     printf( "Nome:           %s %s\n", p->nome, p->
124             sobrenome );
125     printf( "Sexo:           %c\n", p->sexo );
126     printf( "Nascimento:      %02d/%02d/%d\n",
127             p->nascimento.dia, p->nascimento.mes, p->
128             nascimento.ano );
129     printf( "Idade:           %d anos\n", idade );
130     printf( "Altura:           %.1f cm\n", p->altura_cm
131             );
132     printf( "Peso:           %.1f kg\n", p->peso_kg );
133
134     printf( "\n== METRICAS DE SAUDE ==\n" );
135     printf( "IMC:                %.2f (%s)\n",
136             imc, classificarIMC( imc ) );
137     printf( "FC maxima:          %d bpm\n",
138             fcMaxima( idade ) );
139     printf( "FC ideal (50%-85%): %.0f - %.0f bpm\n",
140             ,
141             fcIdealMin( idade ), fcIdealMax( idade ) );
142
143     printf( "\n== TABELA DE IMC (OMS) ==\n" );
144     printf( "  Abaixo do peso:   IMC < 18.5\n" );
145     printf( "  Normal:           18.5 a 24.9\n" );
146     printf( "  Sobrepeso:         25.0 a 29.9\n" );
147     printf( "  Obeso:            IMC >= 30.0\n" );
148
149     printf( "\nLembre: estes valores sao orientativos\n" );
150     printf( "Sempre consulte um profissional de saude\n" );
151 }
```

Exemplo de execução:

Saída do programa

```
===== SISTEMA DE PERFIL DE SAUDE =====

Nome:  Maria
Sobrenome:  Silva
Sexo (M/F): F
Data de nascimento (dia mes ano):  15 6 1990
Altura em cm:  165
Peso em kg:  60

===== PERFIL =====
Nome:  Maria Silva
Sexo:  F
Nascimento:  15/06/1990
Idade:  35 anos
Altura:  165.0 cm
Peso:  60.0 kg

=== METRICAS DE SAUDE ===
IMC: 22.04 (Normal)
FC maxima:  185 bpm
FC ideal (50%-85%):  93 - 157 bpm

=== TABELA DE IMC (OMS) ===
Abaixo do peso:  IMC < 18.5
Normal:  18.5 a 24.9
Sobrepeso:  25.0 a 29.9
Obeso:  IMC >= 30.0
```

Análise didática

Boa prática de programação

Por que struct brilha aqui?

Sem struct, precisaríamos passar 8 ou mais parâmetros para cada função:

```
1  /* SEM struct -- horrivel */
2  void exibir( char *nome, char *sobrenome, char sexo,
3              int dia, int mes, int ano,
4              double altura, double peso );
```

Com struct, encapsulamos tudo:

```
1  /* COM struct -- limpo */
2  void exibir( const PerfilSaude *p );
```

Outros benefícios:

- Adicionar campos (ex.: alergias, medicamentos) não muda assinaturas de funções.
- Passa-se um único ponteiro – eficiente para estruturas grandes.

- Código mais legível e auto-documentado.

Atenção

Aninhamento de struct: note como `PerfilSaude` contém `Data nascimento`. Isso é *composição* – estruturas dentro de estruturas. Para acessar campos aninhados via ponteiro:

```
1 p->nascimento.ano      /* p eh ponteiro; nascimento eh
    subestrutura */
```

A intuição: `->` “deszipa” um ponteiro; `.` acessa membros de uma estrutura já desreferenciada.

Por que isso é “fazer a diferença”?

Registros eletrônicos de saúde

A informatização de registros médicos é uma das maiores transformações da medicina moderna:

Benefícios:

- **Salva vidas em emergências:** médicos acessam histórico completo do paciente – alergias, medicamentos, condições prévias.
- **Evita interações medicamentosas:** sistemas podem alertar automaticamente sobre combinações perigosas.
- **Reduz custos:** elimina exames duplicados, agilizando o atendimento.
- **Pesquisa epidemiológica:** dados agregados (anonimizados) revelam padrões de doenças em populações.

Desafios:

- **Privacidade:** dados de saúde são extremamente sensíveis. LGPD (Brasil) e HIPAA (EUA) regulam estritamente seu uso.
- **Segurança:** sistemas hospitalares são alvos preferenciais de ataques cibernéticos.
- **Interoperabilidade:** sistemas diferentes precisam se comunicar (padrões como HL7 FHIR).
- **Inclusão digital:** nem todos têm acesso a tecnologia.

Você acabou de construir, em escala minimalista, o núcleo de um sistema **Electronic Health Record (EHR)**. Empresas como Epic, Cerner, e a brasileira MV Sistemas movimentam bilhões de reais com esse tipo de software.

Reflexão Final – Capítulo 10

Da memória bruta aos tipos próprios

O Capítulo 10 representa um salto qualitativo em sua jornada com C:

O que mudou?

- Antes: variáveis isoladas, arrays homogêneos, ponteiros.
- Agora: **tipos próprios**, modelando entidades do mundo real.

Conceitos que se tornam possíveis:

- **Modelagem de domínio:** PerfilSaude, Card, Conta, Aluno...
- **Encapsulamento:** função recebe uma estrutura, não 10 parâmetros.
- **Composição:** struct dentro de struct (ex.: Data em Perfil).
- **Manipulação de bits:** controle fino sobre representação binária – essencial em sistemas embarcados, gráficos, criptografia.
- **Uniões:** mesmo espaço, múltiplas interpretações – útil em parsers e drivers.

Próximos passos:

- **Cap. 11:** arquivos – persistir nossas structs em disco.
- **Cap. 12:** ponteiros para struct formam listas encadeadas, filas, pilhas, árvores – as estruturas de dados clássicas.
- **Cap. 13:** pré-processador para macros poderosas com estruturas.

A combinação **struct + typedef + ponteiro** é o tripé sobre o qual se constrói todo o software sério em C. Domine esses três e você terá as ferramentas para projetar sistemas de qualquer complexidade.